

COM 316 ASSIGNMENT

P101/4292G/23 Ephantus Mwago

Question1: Introduction to AI

Reactive agents respond directly to the environment without planning. for example a robot vacuum cleans when it detects dirt, otherwise it moves randomly while **Deliberative agents** build internal models, plan ahead, and reason before acting , for example a self-driving car planning a route.

Simple reactive agent

```
import random

# Environment: 0 = clean, 1 = dirt

environment = [random.choice([0, 1]) for _ in range(10)]
robot_position = 0

print("Initial environment:", environment)

for step in range(10):

    # If there's dirt at current position then clean it

    if environment[robot_position] == 1:

        environment[robot_position] = 0

        print(f"Step {step+1}: Cleaned dirt at position {robot_position}")

    else:
```

```
# Move randomly left or right but stay within bounds

move = random.choice([-1, 1])

robot_position = max(0, min(len(environment) - 1, robot_position +
move))

print(f"Step {step+1}: No dirt, moved to position {robot_position}")

print("Final environment:", environment)
```

Example of my Output

Initial environment: [0, 1, 0, 1, 0, 0, 1, 0, 0, 1]

Step 1: No dirt, moved to position 1

Step 2: Cleaned dirt at position 1

Step 3: No dirt, moved to position 2

Step 4: No dirt, moved to position 3

Step 5: Cleaned dirt at position 3

Step 6: No dirt, moved to position 4

Step 7: No dirt, moved to position 5

Step 8: No dirt, moved to position 6

Step 9: Cleaned dirt at position 6

Step 10: No dirt, moved to position 7

Final environment: [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

- The robot vacuum senses its current position . If dirty, it cleans, otherwise, it moves randomly. No planning, only reaction.

Question 2: Intelligent Agent Design

A **goal-based agent** takes actions to achieve a target, not just react. Here, the goal is to reach **position = 10** starting from **0**.

```
goal = 10
```

```
position = 0
```

```
print(f"Starting at position {position}, goal is {goal}")
```

```
while position != goal:
```

```
    # Decide direction
```

```
    if position < goal:
```

```
        action = "move forward"
```

```
        position += 1
```

```
    else:
```

```
        action = "move backward"
```

```
        position -= 1
```

```
    print(f"Agent {action} , current position = {position}")
```

```
print("Goal reached")
```

Example of my output

Starting at position 0, goal is 10

Agent move forward , current position = 1

Agent move forward , current position = 2

Agent move forward , current position = 3

Agent move forward , current position = 4

Agent move forward , current position = 5

Agent move forward , current position = 6

Agent move forward , current position = 7

Agent move forward , current position = 8

Agent move forward , current position = 9

Agent move forward , current position = 10

Goal reached

Question 3: NLP Word Analysis

```
def word_analysis(paragraph):
```

```
    # Convert to lowercase and split into words
```

```
    words = paragraph.lower().replace('.', '').split()
```

```
    total_words = len(words)
```

```
    print("Total words =", total_words)
```

Count frequencies

```
freq = {}
```

```
for w in words:
```

```
    freq[w] = freq.get(w, 0) + 1
```

Find most frequent word

```
most_frequent = max(freq, key=freq.get)
```

Unique words case-sensitive for display

```
unique = list(set(paragraph.replace('.', '').split()))
```

```
print("Most frequent =", most_frequent)
```

```
print("Unique words =", unique)
```

Example test

```
text = "AI makes machines learn. Machines can learn fast."
```

```
word_analysis(text)
```

my Output

```
Total words = 7
```

```
Most frequent = machines
```

```
Unique words = ['AI', 'makes', 'machines', 'learn', 'can', 'fast']
```

- The code counts total words, tracks frequencies, and extracts unique ones using simple loops and dictionaries.

Question 4:

NLP Tokenization & Sentiment

```
def sentiment_analyzer(sentence):  
    positive = ['good', 'happy', 'great']  
    negative = ['bad', 'sad', 'poor']  
    words = sentence.lower().replace('.', '').split()  
    score = 0  
  
    # Assign sentiment score  
  
    for word in words:  
        if word in positive:  
            score += 1  
        elif word in negative:  
            score -= 1  
  
    # Determine sentiment  
  
    if score > 0:  
        sentiment = "Positive"  
    elif score < 0:
```

```
        sentiment = "Negative"
else:
    sentiment = "Neutral"
print(f"Sentiment Score: {score} , {sentiment}")
```

Example

```
sentiment_analyzer("The movie was good but the ending was bad.")
```

My output

Sentiment Score: 0 , Neutral

- The function checks each word against lists of positive and negative words to calculate the overall sentiment.

Question 5 : AI Reasoning Challenge

```
from collections import deque
```

0 = free path, 1 = obstacle

```
grid = [
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
```

```
[0, 1, 1, 1, 0],  
[0, 0, 0, 0, 0]  
]
```

```
start = (0, 0)
```

```
goal = (4, 4)
```

BFS Search

```
def bfs_path(grid, start, goal):
```

```
    rows, cols = len(grid), len(grid[0])
```

```
    queue = deque([(start, [start])]) # current pos, path so far
```

```
    visited = set([start])
```

```
    while queue:
```

```
        (x, y), path = queue.popleft()
```

```
        if (x, y) == goal:
```

```
            return path # Path found
```

Explore 4 possible directions

```
    for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
```

```
        nx, ny = x + dx, y + dy
```

```
        if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 0 and (nx,
ny) not in visited:
```

```
            visited.add((nx, ny))
```

```
            queue.append(((nx, ny), path + [(nx, ny)]))
```

```
    return None # No path found
```

```
path = bfs_path(grid, start, goal)
```

```
print("Path found:" if path else "No path found.")
```

```
print(path)
```

My output

Path found:

```
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
```

- The agent explores neighboring cells using **Breadth-First Search** (BFS), avoiding obstacles until it reaches the goal.